

Book Management API — Project Requirements

1. Problem Statement

Small libraries and personal book collections often need a simple system to keep track of their books.

Currently, book information may be stored manually in spreadsheets or notebooks. This makes it difficult to:

- Add new books
- Find a particular book
- Update book information
- Remove books
- See all available books
- Keep book information organized

The goal is to build a RESTful Book Management API using Spring Boot and PostgreSQL.

The API will allow users to manage book information through standard CRUD operations.

2. Project Scenario

Imagine a small library called BookNest.

BookNest wants a backend system where its staff can manage their collection of books.

For every book, the system should store information such as:

- Book title
- Author
- Price
- Publication date
- Availability status

A staff member should be able to communicate with the backend through REST APIs.

For example:

“Add a new book.”

The backend receives the request, validates the data, processes it, stores it in PostgreSQL, and returns an appropriate response.

3. Main Goal

Build a backend API that supports:

Create — Add a new book.

Read — View all books or a specific book.

Update — Modify an existing book.

Delete — Remove a book.

4. Book Information

Each book should contain:

ID — Unique identifier

Title — Name of the book

Author — Author's name

Price — Book price

Published Date — Date the book was published

Available — Whether the book is currently available

Created At — When the record was created

Updated At — When the record was last updated

Example book:

ID: 1

Title: Clean Code

Author: Robert C. Martin

Price: 45.00

Published Date: 2008-08-01

Available: true

5. Required API Operations

Create Book

The client should be able to create a new book.

POST /books

Get All Books

The client should be able to retrieve all books.

GET /books

Get One Book

The client should be able to retrieve a specific book using its ID.

GET /books/{id}

Update Book

The client should be able to update an existing book.

PUT /books/{id}

Delete Book

The client should be able to remove a book.

DELETE /books/{id}

6. Basic Validation Requirements

The API should not accept obviously invalid book data.

Title:

- Required
- Cannot be empty

Author:

- Required
- Cannot be empty

Price:

- Required
- Must not be negative

Published Date:

- Should be a valid date

7. Error Requirements

The API should provide meaningful responses when something goes wrong.

Book doesn't exist:

If someone requests GET /books/999 and book 999 doesn't exist, the API should return an appropriate 404 Not Found response.

Invalid data:

If someone tries to create a book with an empty title, the API should return a validation error.

8. Database Requirements

Use PostgreSQL.

The database should contain a table representing books.

Conceptually:

books

id
title
author
price
published_date
available
created_at
updated_at

9. Spring Boot Architecture

The project should follow a basic layered architecture:

Controller
↓
DTO
↓
Service
↓
Repository
↓
Entity
↓
PostgreSQL

Controller — Handles HTTP requests and responses.

DTO — Represents data coming into or going out of the API.

Service — Contains the application's business logic.

Repository — Handles communication with the database through Spring Data JPA.

Entity — Represents the database table in Java.

PostgreSQL — Stores the actual book data.

10. Technologies

Use:

- Java
- Spring Boot
- Spring Web
- Spring Data JPA
- Bean Validation
- PostgreSQL

- Maven
- Git

You do not need Spring Security or authentication for the first version.

11. Project Scope

Version 1 — Basic

Only implement:

- Book CRUD
- PostgreSQL
- DTO
- Validation
- Entity
- Repository
- Service
- Controller
- Basic exception handling

Do NOT add initially:

- Login
- JWT
- Roles
- Admin dashboard
- File uploads
- Microservices
- Docker
- Redis
- Messaging
- Cloud deployment

Those can come later.

12. Example User Scenario

A librarian receives a new book:

“Clean Architecture” by Robert C. Martin, priced at \$40.

The librarian sends the book information to the API.

The system:

Request

↓

Controller
↓
Validate DTO
↓
Service
↓
Repository
↓
Entity
↓
PostgreSQL

The book is stored successfully.

Later, the librarian can:

GET — View books

GET /1 — View book #1

PUT /1 — Update book #1

DELETE /1 — Delete book #1

13. Learning Goals

By completing this project, you should understand the basic Spring Boot backend flow:

Spring Boot project
→ Configuration
→ Beans
→ Dependency Injection
→ Controller
→ DTO
→ Validation
→ Service
→ Repository
→ Entity
→ JPA/Hibernate
→ PostgreSQL
→ CRUD
→ Exception handling

This project is intentionally small. The goal isn't to build a complicated library system; the goal is to understand how a real Spring Boot backend is structured.